Kindergarten API — інструкція для розробників

Цей документ описує все, що зроблено в проєкті станом на зараз: як запустити, як тестувати,
як він влаштований, які бізнес-правила реалізовані, що залишилось доробити.

---

1. Що це за проєкт

Серверна частина (backend, **без frontend**) для керування приватним дитячим садком.
Формат: **RESTful API на Node.js**. Вимоги взято з обговорення в чаті (файл `nodejs.docx`).

Що вміє система:

- облік дітей (2–5 років), батьків, вихователів, помічників, викладачів-предметників;
- групи (один вік у групі, максимум 30 дітей, обов'язкові вихователь і помічник);
- автоматичне складання тижневого розкладу з урахуванням обмежень;
- розрахунок оплати для батьків (навчання, пільги, харчування, оплата за рік наперед);
- розрахунок прибутку (по кожній групі і загалом) та рекомендація оптимальної конфігурації садка;
- ролі та права доступу (адмін, менеджер, батьки, викладачі);
- збереження історії всіх змін (стан «до» і «після»).

*> ВАЖЛИВО. У тексті ТЗ немає відповіді, яку саме **«суперфічу»** (додаткову технічну вимогу:*
> SQL / MongoDB / без БД / 95% покриття тощо) отримала наша команда. Поточна версія — базова:
> дані в пам'яті з опційним збереженням у JSON-файл. Покриття тестами 95% вже виконано.
> Коли суперфіча стане відома — адаптувати шар сховища (див. розділ 6).

---

2. Вимоги та встановлення

- **Node.js 22 або новіший** (розробка і тести проходили на 22; на macOS перевірено на 24).
- npm (ставиться разом з Node).

Встановлення Node на macOS

Варіант А — інсталятор: завантажити LTS з https://nodejs.org і встановити `.pkg`. Після цього **закрити й відкрити термінал заново**.

Варіант Б — через nvm:

```bash
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.8/install.sh | bash
export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
nvm install 24
```

Якщо nvm не підхоплюється в новому терміналі (файл `~/.zshrc` не існував), додати рядки вручну:

```bash
cat >> ~/.zshrc <<'EOF'
export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
EOF
```

Перевірка: `node -v` і `npm -v` мають показати версії.

---

3. Запуск

```bash
cd ~/Documents/"project pnu"/kindergarten   # шлях з пробілом — обов'язково в лапках
npm install
npm start
```

У терміналі з'явиться `Kindergarten API listening on http://localhost:3000`. Це вікно не закривати, поки сервер потрібен. Зупинка: `Ctrl+C`.

Перевірка, що сервер живий (у **другому** терміналі):

```bash
curl http://localhost:3000/health
# {"status":"ok"}
```

Змінні оточення (усі необов'язкові)

Змінна	За замовчуванням	Призначення
`PORT`	`3000`	порт сервера
`DATA_FILE`	не задано	шлях до JSON-файла для збереження даних між запусками. Без нього все живе тільки в пам'яті
`JWT_SECRET`	`dev-secret-change-me`	секрет підпису токенів. **Для будь-чого окрім локальної розробки змінити**
`ADMIN_USER`	`admin`	логін першого адміна
`ADMIN_PASSWORD`	`admin123`	пароль першого адміна

Приклад: `DATA_FILE=data.json PORT=4000 npm start`.

Адмін створюється автоматично при старті, якщо в сховищі ще немає жодного адміна.

---

4. Тестування

```bash
npm test
```

Запускає Jest з покриттям. У `package.json` налаштовано поріг:
statements 95%, lines 95%, functions 95%, branches 90% — якщо покриття впаде нижче, команда завершиться помилкою.

Поточний стан: **52 тести, покриття близько 99.7%**.

Важливо: `npm test` і `npm start` потрібно виконувати **з папки проєкту** (де лежить `package.json`),
інакше буде помилка `ENOENT ... package.json`.

Тести не використовують порт 3000 (supertest піднімає застосунок у пам'яті), тому їх можна
запускати, не зупиняючи сервер.

Структура тестів

- `tests/helpers.js` — `setup()` (чистий застосунок + адмін-токен), `seedKindergarten()`
 (готовий садок: вихователь, помічник, 5 предметників, група 3 роки, 2 батьків, 2 дітей), `yearsAgo(n)` (дата народження для віку n).

- `tests/units.test.js` — юніт-тести: розрахунок віку/оплати, генерація розкладу, сховище, хешування паролів.
- `tests/api.test.js` — інтеграційні тести всіх ендпоінтів, прав доступу та помилок.

Дати народження в тестах рахуються від поточної дати (`yearsAgo`), тому тести не «протухають».

---

5. Швидкий сценарій руками (curl)

Запустити сервер (`npm start`), в іншому терміналі:

```bash
B=http://localhost:3000
TOKEN=$(curl -s -X POST $B/auth/login -H 'Content-Type: application/json' \
 -d '{"username":"admin","password":"admin123"}' | node -pe
'JSON.parse(require("fs").readFileSync(0)).token')
H="Authorization: Bearer $TOKEN"; J='Content-Type: application/json'

# Персонал (id виходять 1, 2, 3...: кожна колекція має свій лічильник)
curl -s -X POST $B/staff -H "$H" -H "$J" -d '{"name":"Olga","type":"educator"}'
# staff id 1
curl -s -X POST $B/staff -H "$H" -H "$J" -d '{"name":"Iryna","type":"assistant"}'
# staff id 2
for s in music english pe wrestling chess; do
 curl -s -X POST $B/staff -H "$H" -H "$J" \
   -d "{\"name\":\"Teacher $s\",\"type\":\"subject\",\"subjects\":[\"$s\"]}"
# staff id 3..7
done

# Група для 3-річних
curl -s -X POST $B/groups -H "$H" -H "$J" \
 -d '{"name":"Sunflowers","age":3,"educatorId":1,"assistantId":2}'

# Батько — це користувач; id беруться з колекції users, де адмін має id 1, тому перший батько = id 2
curl -s -X POST $B/parents -H "$H" -H "$J" -d
'{"username":"mom","password":"pw","name":"Mom"}'

# Дитина (дата народження має давати вік 3 роки на сьогодні!) з пільгою 10%
curl -s -X POST $B/children -H "$H" -H "$J" \
 -d '{"name":"Kid","birthDate":"2023-03-01","parentId":2,"discountPercent":10}'
curl -s -X POST $B/groups/1/children -H "$H" -H "$J" -d '{"childId":1}'
```

```
# Розклад, оплата, прибуток, оптимізація, історія
curl -s -X POST $B/schedule/generate -H "$H"
curl -s $B/children/1/billing -H "$H"
curl -s $B/profit -H "$H"
curl -s $B/optimization -H "$H"
curl -s $B/history -H "$H"

# Дивимось очима батька
PT=$(curl -s -X POST $B/auth/login -H "$J" -d '{"username":"mom","password":"pw"}'
\
 | node -pe 'JSON.parse(require("fs").readFileSync(0)).token')
curl -s $B/children -H "Authorization: Bearer $PT"
curl -s $B/children/1/billing -H "Authorization: Bearer $PT"
```

Зверніть увагу: **id у різних колекціях незалежні** (users, children, groups, staff).
`parentId` у дитини — це id із колекції **users** (ту, де адмін має id 1).
Перевіряйте id у відповідях на створення.

---

6. Архітектура та структура коду

```
kindergarten/
├── package.json          # скрипти start/test, налаштування Jest і порогів покриття
├── README.md             # короткий опис
├── DEVELOPERS.md         # цей документ
├── src/
│   ├── server.js         # точка входу: читає env, створює Store, seedAdmin, слухає порт
│   ├── app.js            # createApp(store): ВСІ маршрути, валідація, права доступу; seedAdmin
│   ├── store.js          # сховище в пам'яті + JSON-файл + історія змін
│   ├── auth.js           # хешування паролів (scrypt), JWT, middleware authenticate/requireRole
│   ├── billing.js        # вік, ціна харчування, розрахунок оплати дитини (чисті функції)
│   ├── schedule.js       # генерація розкладу + представлення для ролей (чисті функції)
│   ├── finance.js        # розрахунок прибутку та оптимізація (працює над Store)
│   ├── config.js         # DEFAULT_SETTINGS (бізнес-параметри) та DAY (сітка дня)
```

```
│   └── errors.js        # HttpError та хелпери
bad/notFound/forbidden/unauthorized/conflict
└── tests/
    ├── helpers.js
    ├── units.test.js
    └── api.test.js
```

Потік запиту

`запит → express.json() → authenticate (JWT) → requireRole(...) → обробник у app.js → Store → відповідь`.
Помилки: обробники кидають `HttpError` (`bad`, `notFound`, ...), глобальний обробник у кінці `app.js`
перетворює їх на JSON `{ "error": "...", "details": ... }`. Невідомі помилки дають 500.

Принципи, яких варто триматись

1. **Чисті функції окремо від HTTP.** Формули (`billing.js`, `schedule.js`) не знають про Express — їх легко тестувати.
2. **`createApp(store)` приймає сховище ззовні** — у тестах кожен тест отримує свіжий `Store` у пам'яті.
3. **Усі зміни даних — тільки через методи `Store`** (`insert/update/remove/updateSettings/setSchedule`),
   бо саме вони пишуть історію змін і зберігають файл. Не змінюйте `store.data` напряму в бізнес-коді.
4. **Права перевіряються на рівні маршруту** (`requireRole`) і додатково на рівні об'єкта
   (`canSeeChild`: чи це дитина цього батька / чи це група цього вихователя).
5. **Налаштування бізнес-правил (ціни, зарплати, ліміти) лежать у `settings`**, а не «зашиті» в код.
   Адмін змінює їх через `PATCH /settings`.

Якщо потрібно замінити сховище (SQL / MongoDB / інше)

Весь доступ до даних іде через клас `Store` (`list`, `get`, `find`, `filter`, `insert`, `update`, `remove`,
`updateSettings`, `setSchedule`) і прямі звернення до `store.data.settings`, `store.data.schedule`,
`store.data.history` в `app.js`/`finance.js`. Це точка заміни: або написати новий клас з тим самим інтерфейсом,
або винести інтерфейс у репозиторій. Зверніть увагу, що зараз методи **синхронні**; для реальної БД
доведеться зробити їх асинхронними (`await`) і оновити обробники та тести.

---

7. Ролі та права

Роль	Як з'являється	Що може
`admin`	автоматично при старті	усе: пільги й фінансові параметри, налаштування, персонал, групи, `/profit`, `/optimization`, `/history`, видалення
`manager`	`POST /managers` (адмін)	реєструвати батьків і дітей, редагувати дітей (крім пільг/плану оплати), призначати в групи, генерувати розклад, бачити налаштування
`parent`	`POST /parents` (адмін/менеджер)	лише **свої** діти: дані, розклад, розраховану оплату
`teacher`	`POST /staff` з `username`+`password`	свій розклад (`GET /schedule`, `/staff/:id/schedule`); групи і діти, де він вихователь/помічник

Токен: `POST /auth/login` → `{ token }`, далі заголовок `Authorization: Bearer <token>`. Термін дії 12 годин.

---

8. Модель даних

Усі записи мають числовий `id` (окремий лічильник для кожної колекції), `createdAt`, після змін — `updatedAt`.

- **users**: `username, name, role (admin|manager|parent|teacher), passwordHash, phone?, staffId?` (staffId — для teacher).
  Пароль ніколи не повертається в API.
- **staff**: `name, type (educator|assistant|subject), subjects[] (лише для subject), salary?`
  Предмети: `music, english, pe, wrestling, chess`. Якщо `salary` не задано — береться `settings.teacherSalary`.
- **groups**: `name, age (2-5), educatorId, assistantId`. Один вихователь/помічник — тільки в одній групі.
- **children**: `name, birthDate, parentId (id користувача-батька), groupId|null, discountPercent (0/10/20/30), paymentPlan (monthly|yearly)`.
  У відповідях додається обчислюване поле `age`.
- **settings** (один об'єкт): див. розділ 10.
- **schedule** (один об'єкт): `{ generatedAt, days: { [groupId]: { Mon|Tue|Wed|Thu|Fri: [16 слотів] } } }`.
- **history**: `{ seq, at, actor, action (create|update|delete|generate), collection, id, before, after }`.

```
---

## 9. Довідник API

Усі тіла запитів — JSON. Позначки: **A** = admin, **M** = manager, **P** = parent, **T** = teacher.

### Авторизація
| Метод | Шлях | Хто | Опис |
|---|---|---|---|
| GET | `/health` | усі | `{status:"ok"}` |
| POST | `/auth/login` | усі | `{username,password}` → `{token,user}` |
| GET | `/auth/me` | будь-хто з токеном | поточний користувач |

### Користувачі
| Метод | Шлях | Хто | Тіло |
|---|---|---|---|
| POST | `/managers` | A | `{username,password,name}` |
| POST | `/parents` | A, M | `{username,password,name,phone?}` |
| GET | `/parents` | A, M | список |

### Персонал
| Метод | Шлях | Хто | Опис |
|---|---|---|---|
| POST | `/staff` | A | `{name,type,subjects?,salary?,username?,password?}`; для `subject` обов'язковий `subjects` (непорожній підмасив допустимих предметів); `username`+`password` разом створюють акаунт викладача |
| GET | `/staff` | A, M | список |
| PATCH | `/staff/:id` | A | `{name?,subjects?,salary?}` |
| DELETE | `/staff/:id` | A | 409, якщо призначений у групу |
| GET | `/staff/:id/schedule` | A, M, T (свій) | розклад викладача по днях |

### Групи
| Метод | Шлях | Хто | Опис |
|---|---|---|---|
| POST | `/groups` | A | `{name,age,educatorId,assistantId}` |
| GET | `/groups` | A, M, T | T бачить лише групи, де він вихователь/помічник; є `childrenCount` |
| PATCH | `/groups/:id` | A | `{name?,educatorId?,assistantId?}` |
| DELETE | `/groups/:id` | A | 409, якщо в групі є діти |
| POST | `/groups/:id/children` | A, M | `{childId}`; перевіряє вік = вік групи та ліміт 30 |
| DELETE | `/groups/:id/children/:childId` | A, M | прибрати дитину з групи |

### Діти
```

Метод	Шлях	Хто	Опис
POST	`/children`	A, M	`{name,birthDate,parentId,discountPercent?,paymentPlan?}`; вік 2-5; пільги ставить лише A
GET	`/children`	усі	відфільтровано за роллю (P — свої, T-вихователь — своєї групи)
GET	`/children/:id`	усі (з перевіркою доступу)	
PATCH	`/children/:id`	A, M	`discountPercent` і `paymentPlan` — тільки A
DELETE	`/children/:id`	A	
GET	`/children/:id/billing`	A, M, P (своя)	query: `months` (1-12), `plan` (monthly|yearly), `workDays` (1-31)
GET	`/children/:id/schedule`	усі з доступом	розклад групи дитини

Розклад
Метод	Шлях	Хто	Опис
POST	`/schedule/generate`	A, M	перегенерує розклад усіх груп. **201** — без конфліктів, **207** — є конфлікти (`conflicts[]`), **409** — немає груп
GET	`/schedule`	усі	A/M — усі групи; P — групи своїх дітей; T — його уроки по днях

Фінанси
Метод	Шлях	Хто	Опис
GET	`/profit`	A	прибуток по групах і загалом (на місяць)
GET	`/profit/groups/:id`	A	прибуток однієї групи
GET	`/optimization`	A	рекомендована конфігурація (кімнати, викладачі, прибуток, окупність)

Налаштування та історія
Метод	Шлях	Хто	Опис
GET	`/settings`	A, M	усі параметри
PATCH	`/settings`	A	змінює числові параметри та `allowedDiscounts`; `subjects`, `minAge`, `maxAge` лише читаються
GET	`/history`	A	query: `collection`, `id` — фільтри

Коди відповідей
`200/201` успіх, `204` видалено, `207` розклад згенеровано з конфліктами, `400` невалідні дані,
`401` немає/невалідний токен або логін, `403` недостатньо прав, `404` не знайдено, `409` порушення бізнес-правила (вік не збігається, група повна, дублікат тощо).

---

10. Бізнес-правила та формули

Налаштування за замовчуванням (`src/config.js`, змінюються через `PATCH /settings`)

Параметр	Значення	Походження
`baseTuition`	15000 грн/міс	ТЗ
`allowedDiscounts`	0, 10, 20, 30 %	ТЗ
`prepayDiscountPercent`	12 %	ТЗ (оплата за рік наперед)
`mealPriceForAge2` / `mealPriceStepPerYear`	300 / +100 грн за день	ТЗ (приклад з чату)
`teacherSalary`	40000 грн	ТЗ (використовується для вихователів, помічників і викладачів)
`managerSalary`	80000 грн	ТЗ
`maxGroupSize`	30	пропозиція з чату
`workDaysPerMonth`	22	**припущення**
`mealCostPercent`	100	**припущення**: харчування без маржі (собівартість = ціні)
`roomRentPerMonth`	0	**припущення**: у ТЗ немає
`managersOnPayroll`	1	**припущення**
`teacherMaxLessonsPerDay`	8	**припущення**

Оплата (`calcBilling`)

```
навчання_після_пільги = baseTuition * (1 - discountPercent/100)
навчання_за_місяць    = навчання_після_пільги * (1 - 12%, якщо plan = yearly)
харчування_за_день    = 300 + (вік - 2) * 100
харчування_за_місяць  = харчування_за_день * workDaysPerMonth
разом = (навчання_за_місяць + харчування_за_місяць) * кількість_місяців
```

- Пільга діє тільки на навчання, на харчування знижок немає.
- Знижка 12% застосовується **після** пільги (множаться, не додаються).
- При `plan = yearly` період завжди 12 місяців.
- Вік — повні роки на сьогодні.

Прибуток (`calcProfit`, на місяць)

- Враховуються тільки діти, які **призначені в групу**.
- Дохід групи = навчання (з пільгами та планом оплати) + харчування.
- Витрати групи: зарплата вихователя + помічника + оренда кімнати. Собівартість харчування = `mealCostPercent` від виручки за харчування.

- **Спільні витрати** (всі викладачі-предметники + менеджери) розподіляються між групами пропорційно кількості дітей.
- `total.profit = дохід - собівартість харчування - прямі витрати - спільні витрати`.
 Якщо груп немає, спільні витрати (зарплата менеджера) все одно дають мінус.

Оптимізація (`optimize`)

- Враховує **всіх** дітей (не лише призначених у групи), групує за віком.
- Кількість груп для віку = `ceil(дітей / maxGroupSize)` — це мінімальна кількість кімнат.
- Потрібні викладачі на предмет = `max(1, ceil(груп / teacherMaxLessonsPerDay))`;
 показує, скільки ще треба найняти порівняно з наявними (`toHire`, `extraToHire`).
- `breakEvenChildrenPerGroup` — скільки дітей треба в групі, щоб покрити вихователя + помічника + кімнату.
- Повертає очікуваний прибуток і прапорець `profitable`.

Розклад (`generateSchedule`)

Сітка дня: **08:00–16:00, 16 слотів по 30 хв**.

Слоти	Час	Що
0	08:00	сніданок
8	12:00	обід
9–10	12:30–13:30	сон (1 година)
13	14:30	полуденок
решта (11 слотів)		вільні слоти для занять

Алгоритм (жадібний, детермінований): для кожного дня і кожної групи по черзі — 5 занять (по одному на кожен предмет); порядок предметів ротується залежно від дня і номера групи;
бажані слоти розподіляються рівномірно по доступних слотах (`spread`); для кожного заняття береться найближчий
до бажаного слот, де є вільний викладач цього предмета (меншим навантаженням, потім меншим id),
і не перевищено `teacherMaxLessonsPerDay`.

Гарантії: викладач ніколи не веде дві групи в один слот; у кожної групи щодня всі 5 предметів
(якщо вистачає викладачів).
Конфлікти повертаються в `conflicts[]`: `no_teacher` (немає жодного викладача предмета) та
`no_free_slot` (не вдалося знайти слот).

---

11. Як проектувати і додавати нові функції

1. **Почніть із правила.** Якщо це бізнес-формула — додайте/змініть чисту функцію в `billing.js`/`schedule.js`/`finance.js`
   і напишіть юніт-тест у `tests/units.test.js` з конкретними числами.
2. **Новий параметр** (ціна, ліміт) — додайте в `DEFAULT_SETTINGS` у `config.js` і в список `numeric` у `PATCH /settings` (`app.js`).
3. **Новий ендпоінт** — маршрут у `app.js`, у порядку: `authenticate` → `requireRole` → валідація (`bad(...)`) →
   перевірка існування (`notFound`) → бізнес-правила (`conflict`) → зміна через `store.*` → відповідь.
   Для доступу до об'єктів конкретного користувача використовуйте перевірку на рівні об'єкта, як `canSeeChild`.
4. **Нова сутність** — додайте назву в `COLLECTIONS` у `store.js`; історія та лічильники id запрацюють автоматично.
5. **Тести — обов'язково.** Інтеграційний тест у `tests/api.test.js`: щасливий шлях, невалідні дані, чужий доступ (403),
   неіснуючий об'єкт (404). Перевірте `npm test` — поріг покриття 95% не повинен просісти.
6. **Стиль:** `'use strict'`, CommonJS (`require`), чисті функції окремо від Express, помилки через `HttpError`.

Перелік рішень, що вже прийняті (не ламати без обговорення)

- Групи мають фіксований вік; вік дитини перевіряється **при призначенні** в групу.
- Фінансові параметри дитини (пільга, план оплати) змінює тільки адмін; менеджер реєструє дитину без пільг.
- Усі зміни логуються в `history` з актором (`роль:логін`) та станами до/після.
- Паролі зберігаються як `scrypt`-хеш з сіллю.

---

12. Відомі обмеження та що ще треба зробити

1. **Суперфіча команди невідома** — уточнити у викладача й адаптувати сховище/архітектуру (див. розділ 6).
2. **Тимчасова пауза відвідування** (відпустка): у ТЗ немає відповіді, як рахувати оплату — **не реалізовано**. Потрібне уточнення.
3. **Історія платежів** і змін вартості: «бажано» за ТЗ. Зараз є історія змін даних (`/history`), але немає окремої сутності «платіж».
   Оплата розраховується, але не фіксується як факт.
4. **Старіння дітей:** вік змінюється з часом, а дитина залишається в групі. Автоматичного переводу в старшу групу немає.

5. **Розклад** — жадібна евристика, а не оптимальний розв'язок. Не оптимізує «вікна» між заняттями і не враховує побажання батьків.
  Генерація завжди повністю перезаписує попередній розклад.
6. **Зберігання у JSON-файлі** синхронне і не підходить для навантаження чи кількох процесів. Для реального використання — БД.
7. **Безпека:** змінити `JWT_SECRET` і пароль адміна; немає rate limiting, немає зміни/скидання пароля, немає refresh-токенів.
8. **Прибуток** рахується помісячно; річна оплата враховується як місячний еквівалент зі знижкою. `/profit` не враховує дітей без групи.
9. **Немає пагінації** у списках, немає OpenAPI/Swagger-документації (корисно додати для вимоги «RESTful API»).
10. Допущення з розділу 10 (22 робочі дні, оренда 0, харчування без маржі та ін.) потрібно підтвердити з «клієнтом».

---

13. Типові проблеми

Симптом	Причина / рішення
`zsh: command not found: npm`	не встановлено Node або термінал відкритий до встановлення. Встановити Node (розділ 2), відкрити новий термінал
`zsh: command not found: nvm`	nvm не підключено. Виконати `export NVM_DIR=...` рядки з розділу 2
`ENOENT ... package.json`	команда `npm` запущена не з папки проєкту. `cd ~/Documents/"project pnu"/kindergarten`
`cd: no such file or directory`	шлях з пробілом треба брати в лапки: `"project pnu"`
`EADDRINUSE` (порт зайнятий)	сервер уже запущений в іншому терміналі, або змінити порт: `PORT=4000 npm start`
`401 Unauthorized`	немає/прострочений токен — залогінитись знову (токен живе 12 годин)
`409` при призначенні дитини в групу	вік дитини не збігається з віком групи або група заповнена (30)
`400 parentId must reference an existing parent`	`parentId` — це id **користувача-батька** (колекція users, адмін має id 1), а не id із персоналу
Розклад повертає `207`	немає викладача для якогось предмета або вичерпано ліміт занять викладача
У папці проєкту є папка з назвою `~`	випадковий артефакт розпакування, її можна видалити